

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Assignment : Exploring Synchronization Tools and Their Applications in Operating Systems Operating Systems : CSE 4501

<u>Student ID: 210042112</u>

Fifth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

April 28, 2025

Abstract

The purpose of this assignment is to understand synchronization tools, their significance in operating systems, and their implementation in solving concurrency problems.

Contents

1	Introduction	4
1.1	Essence of Synchronization	4
2	Understanding Synchronization Tools	4
2.1	Semaphores	4
2.1.1	Purpose of Semaphores	5
2.1.2	Real World Applications	5
2.1.3	Reader-Writer with Semaphores	5
2.2	Mutexes	6
2.2.1	How Mutexes Differ from Semaphores	6
2.2.2	Role in Preventing Simultaneous Access	6
2.2.3	Shared Counter with Mutex	7
2.2.4	Real World Applications	7
2.3	Monitors	7
2.3.1	Purpose of Monitors	7
2.3.2	Components of a monitor	7
2.3.3	Monitor for producer-consumer	7
2.3.4	Real World Applications	8
2.3.5	Implementation Considerations	8
3	Implementing Synchronization Tools: Producer-Consumer Problem with Semaphores	8
4	Analyzing Concurrency Issues	12
4.1	Deadlock	12
4.2	Race Condition	12
4.3	Starvation	12
4.4	How Synchronization Tools Prevent Concurrency Issues	12
4.4.1	Preventing Race Conditions	13
4.4.2	Preventing Deadlocks	13
4.4.3	Preventing Starvation	13
4.5	Scenarios Where Concurrency Issues Persist Despite Synchronization Tools	14
4.5.1	Deadlock Despite Synchronization	14
4.5.2	Race Condition Despite Synchronization	14
4.5.3	Starvation Despite Synchronization	15
5	Case Study Comparison	15
5.1	Synchronization in Linux	15
5.1.1	Concurrency handling	16
5.1.2	Strengths and Challenges	16
5.2	Synchronization in Windows	16
5.2.1	Concurrency handling	17
5.2.2	Strengths and Challenges	17
5.3	Key Differences	17
5.4	Example: Producer-Consumer Problem	18
6	Visualization	19
7	Conclusion	20

1 Introduction

Synchronization orchestrates the harmonious interplay of processes, ensuring they execute with precision and coherence. It is the art of coordinating concurrent tasks to prevent conflicts, maintain data integrity, and optimize system performance. Without synchronization, the chaos of unsynchronized access to shared resources could lead to corrupted data, erratic behavior, or system failures. Its importance lies in enabling seamless collaboration among processes, safeguarding data consistency, and ensuring efficient resource utilization in the ecosystem of a modern operating system.

1. **Critical Section:** A segment of code where a process accesses a shared resource, such as a variable or file, demanding exclusive control. Like a guarded vault, the critical section allows only one process at a time to enter, preventing simultaneous modifications that could fracture data integrity.
2. **Deadlock:** A paralyzing standoff where two or more processes, each clutching a resource, wait indefinitely for the others to release what they need. This impasse halts progress, akin to travelers at a crossroads, each refusing to yield. Deadlock prevention and resolution are vital to maintaining system fluidity.
3. **Semaphore:** A versatile signaling mechanism, a semaphore is an integer variable used to control access to shared resources. Acting as a gatekeeper, it permits or restricts processes based on its value, enabling coordination through operations like wait (decrement) and signal (increment). Semaphores come in binary (for mutual exclusion) and counting (for resource management) flavors.
4. **Monitor:** An elegant abstraction that encapsulates shared resources and their associated operations within a single, synchronized construct. Monitors ensure that only one process can execute within them at a time, using condition variables to manage waiting and signaling. They provide a structured, high-level approach to synchronization, reducing the risk of errors.

1.1 Essence of Synchronization

Synchronization is the linchpin of reliable multiprocessing, ensuring that data remains consistent and processes cooperate without stepping on each other's toes. By managing access to critical sections, averting deadlocks, and leveraging tools like semaphores and monitors, operating systems maintain order in the face of concurrency's complexity. This delicate balance empowers systems to deliver robust, efficient, and predictable performance, underpinning the seamless functionality we expect from modern computing environments.

2 Understanding Synchronization Tools

2.1 Semaphores

Semaphores are powerful synchronization tools used to manage access to shared resources and coordinate process execution. A semaphore is an integer variable manipulated through two atomic operations: **wait()** (which decrements the semaphore value and may block the process if the value becomes negative) and **signal()** (which increments the value and may wake a waiting process). Semaphores serve to prevent race conditions and ensure orderly access to resources.

Types of semaphores

1. Binary Semaphore:

- A binary semaphore restricts its value to 0 or 1, functioning as a lock to enforce mutual exclusion.
- Purpose: Ensures that only one process can access a critical section at a time, similar to a mutex.
- Example: In a database system, a binary semaphore might control access to a shared table, ensuring that only one process updates it at a time to prevent data corruption.

2. Counting Semaphore:

- A counting semaphore allows its value to range over an unrestricted domain, typically used to manage a pool of resources.
- Purpose: Controls access to a finite number of resources, allowing multiple processes to proceed as long as resources are available.
- Example: In a print server managing multiple printers, a counting semaphore initialized to the number of printers (e.g., 3) allows up to three print jobs to proceed concurrently, blocking additional jobs until a printer is freed.

2.1.1 Purpose of Semaphores

Semaphores address two primary synchronization challenges:

1. **Mutual Exclusion:** Ensuring exclusive access to a critical section.
2. **Condition Synchronization:** Coordinating processes to execute in a specific order or when certain conditions are met (e.g., a resource becomes available).

2.1.2 Real World Applications

- **Operating Systems:** Semaphores manage access to shared memory in multiprocessor systems, ensuring data consistency.
- **Network Servers:** In web servers, counting semaphores limit concurrent connections to prevent overload.
- **Producer-Consumer Problem:** Semaphores synchronize producers (adding items to a buffer) and consumers (removing items), ensuring the buffer neither overflows nor underflows. For instance, a binary semaphore ensures exclusive buffer access, while a counting semaphore tracks available slots.

2.1.3 Reader-Writer with Semaphores

Consider a shared database accessed by multiple reader and writer processes. Readers can access the database concurrently, but writers need exclusive access to prevent data inconsistencies. A binary semaphore ensures mutual exclusion for writers, and a counting semaphore tracks the number of active readers.

```

1 semaphore mutex = 1;      // Binary semaphore for mutual exclusion of reader count
2 semaphore db = 1;        // Binary semaphore for database access
3 int reader_count = 0;    // Tracks number of active readers
4
5 void reader() {
6     while (true) {
7         wait(&mutex);      // Lock to update reader_count
8         reader_count++;
9         if (reader_count == 1) { // First reader locks database
10            wait(&db);
11        }
12        signal(&mutex);    // Release reader_count lock
13        read_database();   // Perform read operation
14        wait(&mutex);      // Lock to update reader_count
15        reader_count--;
16        if (reader_count == 0) { // Last reader unlocks database
17            signal(&db);
18        }
19        signal(&mutex);    // Release reader_count lock
20    }
21 }
22
23 void writer() {

```

```

24 while (true) {
25     wait(&db);           // Lock database for exclusive access
26     write_database();    // Perform write operation
27     signal(&db);        // Release database lock
28 }
29 }

```

In this example:

- The **mutex** semaphore protects the **reader_count** variable, ensuring atomic updates.
- The **db** semaphore ensures that writers have exclusive access and that readers only proceed when no writer is active.
- Multiple readers can read simultaneously, but a writer blocks all readers and other writers until it completes.

2.2 Mutexes

A mutex (short for mutual exclusion) is a synchronization primitive specifically designed to prevent simultaneous access to a shared resource. A mutex is a locking mechanism that allows only one thread or process to enter a critical section at a time. It has two states: **locked (0)** and **unlocked (1)**, and supports two operations: **lock()** (acquire the mutex, blocking if already locked) and **unlock()** (release the mutex, allowing another process to acquire it).

2.2.1 How Mutexes Differ from Semaphores

1. Scope:

- A mutex is strictly for mutual exclusion, ensuring only one thread accesses a critical section.
- Semaphores (especially counting semaphores) can handle both mutual exclusion and condition synchronization, such as managing multiple resources.

2. Ownership:

- Mutexes often enforce ownership, meaning only the thread that locked the mutex can unlock it. This prevents erroneous releases by other threads.
- Semaphores do not enforce ownership; any process can perform a `signal()` operation, making them more flexible but prone to misuse.

3. Complexity:

- Mutexes are simpler, designed for straightforward locking scenarios.
- Semaphores are more versatile but require careful management to avoid errors like deadlocks or starvation.

2.2.2 Role in Preventing Simultaneous Access

Mutexes protect critical sections by ensuring exclusive access. For example, in a multithreaded application updating a shared counter, a mutex prevents race conditions by allowing only one thread to modify the counter at a time.

2.2.3 Shared Counter with Mutex

```
1 mutex_t counter_mutex = MUTEX_INITIALIZER;
2 int shared_counter = 0;
3
4 void increment_counter() {
5     lock(&counter_mutex);
6     shared_counter++;
7     unlock(&counter_mutex);
8 }
```

2.2.4 Real World Applications

- **Thread Synchronization:** In a web browser, mutexes protect shared data structures (e.g., a cache) accessed by multiple rendering threads.
- **File Access:** In a file system, a mutex ensures that only one process writes to a file at a time, preserving data integrity.

2.3 Monitors

A monitor is a high-level synchronization construct that encapsulates shared resources, their operations, and synchronization logic within a single entity. A monitor ensures mutual exclusion by allowing only one process to execute its procedures at a time. It uses condition variables to manage additional synchronization, supporting two operations:

- **wait():** Suspends the calling process and releases the monitor's lock, placing the process in a queue.
- **signal():** Resumes one suspended process waiting on the condition, if any.

2.3.1 Purpose of Monitors

Monitors simplify synchronization by:

- Providing implicit mutual exclusion, reducing the risk of programming errors.
- Supporting condition synchronization through condition variables, allowing processes to wait for specific states (e.g., a resource becoming available).

2.3.2 Components of a monitor

- **Shared Resources:** Data structures or variables accessed within the monitor.
- **Procedures:** Functions that operate on the shared resources, executed with mutual exclusion.
- **Condition Variables:** Used to pause and resume processes based on specific conditions.

2.3.3 Monitor for producer-consumer

A monitor for the producer-consumer problem encapsulates the buffer and its operations, using condition variables to manage full and empty states.

```
1 monitor producer_consumer {
2     int buffer[N];
3     int count = 0, in = 0, out = 0;
4     condition full, empty;
5
6     procedure produce(item) {
```

```

7      if (count == N) wait(full); // Wait if buffer is full
8      buffer[in] = item;
9      in = (in + 1) % N;
10     count++;
11     signal(empty); // Signal consumers that buffer is not empty
12 }
13
14 procedure consume() {
15     if (count == 0) wait(empty); // Wait if buffer is empty
16     item = buffer[out];
17     out = (out + 1) % N;
18     count--;
19     signal(full); // Signal producers that buffer is not full
20     return item;
21 }
22 }

```

2.3.4 Real World Applications

- **Java Synchronization:** Java's *synchronized* methods and blocks implement monitor-like behavior, ensuring thread-safe access to shared objects. For instance, a banking application might use a monitor to manage account transactions, preventing concurrent modifications.
- **Operating System Kernels:** Monitors can manage access to kernel data structures, such as process tables, ensuring safe updates in a multitasking environment.

2.3.5 Implementation Considerations

Monitors are typically implemented by the programming language or operating system. For example:

- **Java:** Provides built-in monitors via *synchronized* keywords and `wait()/notify()` methods.
- **Pthreads:** Uses mutexes and condition variables to emulate monitors in C.
- The monitor's mutual exclusion is enforced by a hidden lock, and condition variables manage waiting queues.

3 Implementing Synchronization Tools: Producer-Consumer Problem with Semaphores

```

1  #include <pthread.h>
2  #include <semaphore.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <unistd.h>
6
7
8  // Define constants for buffer size and number of producers/consumers
9  #define BUFFER_SIZE 5
10 #define NUM_PRODUCERS 2
11 #define NUM_CONSUMERS 2
12 #define MAX_ITEMS 10 // Maximum items each producer will produce
13
14 // Shared buffer and indices
15 int buffer[BUFFER_SIZE];
16 int in = 0; // Index where producer adds next item

```



```

17 int out = 0; // Index where consumer removes next item
18 int item_count = 0; // Tracks number of items in buffer
19
20 // Semaphores for synchronization
21 sem_t mutex; // Binary semaphore for mutual exclusion
22 sem_t empty; // Counting semaphore for empty slots
23 sem_t full; // Counting semaphore for filled slots
24
25 // Counter for total items produced (to terminate program)
26 int total_produced = 0;
27
28 // Producer function
29 void* producer(void* arg) {
30     int producer_id = *(int*)arg;
31     int items_produced = 0;
32
33     while (items_produced < MAX_ITEMS) {
34         // Produce an item (simulated by a unique item number)
35         int item = producer_id * 100 + items_produced + 1;
36
37         // Wait for an empty slot in the buffer
38         // Decrements empty; blocks if no empty slots (empty == 0)
39         sem_wait(&empty);
40
41         // Acquire mutual exclusion to access shared buffer
42         // Ensures only one thread modifies buffer at a time
43         sem_wait(&mutex);
44
45         // Critical section: Add item to buffer
46         buffer[in] = item;
47         in = (in + 1) % BUFFER_SIZE; // Circular buffer
48         item_count++;
49         printf("Producer %d produced item %d, buffer count: %d\n", producer_id,
50             item, item_count);
51         total_produced++;
52
53         // Release mutual exclusion
54         // Allows other producers or consumers to access buffer
55         sem_signal(&mutex);
56
57         // Signal that a new item is available
58         // Increments full, potentially waking a waiting consumer
59         sem_signal(&full);
60
61         items_produced++;
62
63         // Simulate variable production time
64         usleep(rand() % 500000); // Sleep 0-0.5 seconds
65     }
66
67     return NULL;
68 }
69
70 // Consumer function
71 void* consumer(void* arg) {
72     int consumer_id = *(int*)arg;
73
74     while (1) {
75         // Wait for a filled slot in the buffer

```

```

75 // Decrements full; blocks if no items (full == 0)
76 sem_wait(&full);
77
78 // Acquire mutual exclusion to access shared buffer
79 sem_wait(&mutex);
80
81 // Critical section: Remove item from buffer
82 int item = buffer[out];
83 out = (out + 1) % BUFFER_SIZE; // Circular buffer
84 item_count--;
85 printf("Consumer %d consumed item %d, buffer count: %d\n", consumer_id,
item, item_count);
86
87 // Check if all items have been produced and buffer is empty
88 if (total_produced >= NUM_PRODUCERS * MAX_ITEMS && item_count == 0) {
89     sem_signal(&mutex); // Release mutex
90     sem_signal(&empty); // Ensure no deadlock
91     break; // Exit if no more items will be produced
92 }
93
94 // Release mutual exclusion
95 sem_signal(&mutex);
96
97 // Signal that an empty slot is available
98 // Increments empty, potentially waking a waiting producer
99 sem_signal(&empty);
100
101 // Simulate variable consumption time
102 usleep(rand() % 500000); // Sleep 0-0.5 seconds
103 }
104
105 return NULL;
106 }
107
108 int main() {
109     // Seed random number generator for variable sleep times
110     srand(time(NULL));
111
112     // Initialize semaphores
113     // mutex initialized to 1 for binary semaphore (mutual exclusion)
114     sem_init(&mutex, 0, 1);
115     // empty initialized to BUFFER_SIZE (all slots initially empty)
116     sem_init(&empty, 0, BUFFER_SIZE);
117     // full initialized to 0 (no items initially in buffer)
118     sem_init(&full, 0, 0);
119
120     // Create thread arrays for producers and consumers
121     pthread_t producer_threads[NUM_PRODUCERS];
122     pthread_t consumer_threads[NUM_CONSUMERS];
123     int producer_ids[NUM_PRODUCERS];
124     int consumer_ids[NUM_CONSUMERS];
125
126     // Create producer threads
127     for (int i = 0; i < NUM_PRODUCERS; i++) {
128         producer_ids[i] = i + 1;
129         pthread_create(&producer_threads[i], NULL, producer, &producer_ids[i]);
130     }
131
132     // Create consumer threads

```

```

133     for (int i = 0; i < NUM_CONSUMERS; i++) {
134         consumer_ids[i] = i + 1;
135         pthread_create(&consumer_threads[i], NULL, consumer, &consumer_ids[i]);
136     }
137
138     // Join producer threads
139     for (int i = 0; i < NUM_PRODUCERS; i++) {
140         pthread_join(producer_threads[i], NULL);
141     }
142
143     // Join consumer threads
144     for (int i = 0; i < NUM_CONSUMERS; i++) {
145         pthread_join(consumer_threads[i], NULL);
146     }
147
148     // Destroy semaphores
149     sem_destroy(&mutex);
150     sem_destroy(&empty);
151     sem_destroy(&full);
152
153     printf("Simulation complete. All items produced and consumed.\n");
154     return 0;
155 }

```

1. Semaphore used:

- **mutex:** A binary semaphore (initialized to 1) ensures mutual exclusion when accessing the shared buffer. It prevents simultaneous modifications by producers and consumers, protecting critical sections where *in*, *out*, *item_count*, and *buffer* are updated.
- **empty:** A counting semaphore (initialized to *BUFFER_SIZE*) tracks available empty slots in the buffer. Producers wait on this semaphore to ensure the buffer isn't full before adding an item.
- **full:** A counting semaphore (initialized to 0) tracks filled slots. Consumers wait on this semaphore to ensure the buffer isn't empty before removing an item.

2. Producer Synchronization:

- **Step 1:** *sem_wait(&empty)* ensures there's an empty slot. If the buffer is full (*empty == 0*), the producer blocks.
- **Step 2:** *sem_wait(&mutex)* acquires exclusive access to the buffer, preventing other threads from modifying shared variables.
- **Step 3:** The producer adds an item to *buffer[in]*, updates *in* and *item_count*, and increments *total_produced*.
- **Step 4:** *sem_signal(&mutex)* releases the lock, allowing other threads to access the buffer.
- **Step 5:** *sem_signal(&full)* increments the *full* semaphore, signaling that a new item is available, potentially waking a waiting consumer.

3. Consumer Synchronization

- **Step 1:** *sem_wait(&full)* ensures there's an item to consume. If the buffer is empty (*full == 0*), the consumer blocks.
- **Step 2:** *sem_wait(&mutex)* acquires exclusive access to the buffer.
- **Step 3:** The consumer removes an item from *buffer[out]*, updates *out* and *item_count*, and checks if the simulation should terminate (all items produced and buffer empty).
- **Step 4:** *sem_signal(&mutex)* releases the lock.

- **Step 5:** *sem_signal(&empty)* increments the empty semaphore, signaling that a slot is free, potentially waking a waiting producer.

4. Key Features

- **Bounded Buffer:** The buffer is circular with a fixed size (*BUFFER_SIZE = 5*), preventing overflow or underflow through the empty and full semaphores.
 - **Termination Logic:** Producers stop after producing *MAX_ITEMS* each. Consumers check *total_produced* and *item_count* to exit when all items are consumed and no more will be produced.
 - **Random Delays:** *usleep(rand() % 500000)* simulates variable processing times, making the output more realistic and testing synchronization under different timing conditions.
 - **Thread Safety:** The **mutex** semaphore ensures atomic updates to shared variables, preventing race conditions.
5. **Output** The program prints each production and consumption event, showing the producer/consumer ID, item number, and current buffer count. This helps verify that the buffer never exceeds *BUFFER_SIZE* or goes below 0, and that items are produced and consumed in a synchronized manner.

4 Analyzing Concurrency Issues

4.1 Deadlock

Deadlock occurs when a set of processes are blocked indefinitely, each holding at least one resource and waiting for another resource that is held by another process in the set. This creates a circular wait where no process can proceed. Operating System Concepts describes deadlock using the four necessary conditions: mutual exclusion, hold and wait, no preemption, and circular wait.

Example: Two processes, P1 and P2, each hold a file (F1 and F2, respectively) and request the other's file. P1 waits for F2, and P2 waits for F1, resulting in a deadlock.

4.2 Race Condition

A race condition arises when multiple processes or threads access a shared resource concurrently, and the outcome depends on the unpredictable order of their execution. This can lead to inconsistent or corrupted data if at least one access is a write operation.

Example: Two threads increment a shared counter (*counter++*). If both read the counter's value (e.g., 5) simultaneously, increment it, and write back (6), the final value may be 6 instead of the expected 7 due to interleaved execution.

4.3 Starvation

Starvation occurs when a process is perpetually denied access to a resource it needs to proceed, often because other processes with higher priority or favorable scheduling continuously preempt it. Unlike deadlock, the process is not blocked but is unable to make progress.

Example: In a priority-based scheduling system, a low-priority process needing CPU time may be indefinitely delayed if high-priority processes keep arriving.

4.4 How Synchronization Tools Prevent Concurrency Issues

Synchronization tools—semaphores, mutexes, and monitors—provide structured mechanisms to coordinate access to shared resources, mitigating concurrency issues.

4.4.1 Preventing Race Conditions

- **Mechanism:** Race conditions are prevented by ensuring mutual exclusion, where only one process or thread can access a critical section (a segment of code accessing shared resources) at a time.
- **Synchronization Tools:**
 - **Mutexes:** A mutex locks a critical section, allowing only one thread to execute it. For example, in a shared counter scenario, a mutex ensures that *counter++* is atomic, preventing interleaved reads and writes.
 - **Semaphores:** Binary semaphores enforce mutual exclusion similarly to mutexes. Counting semaphores can also coordinate access to multiple resources, ensuring consistent updates. In the producer-consumer problem, a binary semaphore (mutex) protects buffer modifications.
 - **Monitors:** Monitors inherently provide mutual exclusion by allowing only one process to execute their procedures at a time. Condition variables ensure that processes wait for appropriate conditions, avoiding inconsistent states.
- **Example:** In a banking system, a monitor encapsulating an account's balance ensures that concurrent withdrawals and deposits are serialized, preventing race conditions that could result in incorrect balances.

4.4.2 Preventing Deadlocks

- **Mechanism:** Deadlock is mitigated by breaking one or more of the four necessary conditions (mutual exclusion, hold and wait, no preemption, circular wait). Synchronization tools, when used with careful design, help avoid circular waits or enable deadlock detection and recovery.
- **Synchronization Tools:**
 - **Semaphores:** Proper ordering of semaphore operations can prevent circular waits. For instance, in a resource allocation system, requiring processes to acquire resources in a fixed order (e.g., always lock resource A before B) eliminates circular wait.
 - **Mutexes:** Mutexes with timeout mechanisms or ordered locking protocols reduce deadlock risk. For example, a database system might enforce that all transactions lock tables in a predefined order.
 - **Monitors:** Monitors simplify deadlock prevention by centralizing resource access and using condition variables to manage waiting. By encapsulating resource logic, monitors reduce the chance of hold-and-wait scenarios.
- **Example:** In the dining philosophers problem, a monitor can manage chopstick allocation, ensuring philosophers pick up both chopsticks atomically or wait, preventing a circular wait.

4.4.3 Preventing Starvation

- **Mechanism:** Starvation is addressed by ensuring fairness in resource allocation, often through scheduling policies or condition variable signaling that prioritize waiting processes.
- **Synchronization Tools:**
 - **Semaphores:** Semaphores can be paired with fair scheduling. For example, in the reader-writer problem, a semaphore can prioritize writers after a certain number of readers to prevent writer starvation.
 - **Mutexes:** While mutexes focus on mutual exclusion, fair implementations (e.g., FIFO-based locking in POSIX mutexes) reduce starvation by granting locks to threads in the order they requested them.

- **Monitors:** Monitors use condition variables with fair signaling policies (e.g., signaling the longest-waiting process) to ensure that no process is indefinitely delayed.
- **Example:** In a print server, a monitor managing printer access can use a condition variable to signal waiting print jobs in a first-come, first-served order, preventing any job from being starved.

4.5 Scenarios Where Concurrency Issues Persist Despite Synchronization Tools

Even with synchronization tools, improper design or implementation can lead to concurrency issues.

4.5.1 Deadlock Despite Synchronization

In a database system, two transactions use mutexes to lock tables but acquire them in different orders. Transaction T1 locks Table A and waits for Table B, while T2 locks Table B and waits for Table A, creating a circular wait.

WHY? The mutexes enforce mutual exclusion but do not enforce a global locking order, violating the prevention of circular wait. This is a common issue when developers fail to standardize resource acquisition protocols.

```

1  void transaction1() {
2      pthread_mutex_lock(&mutex_A); // Lock Table A
3      sleep(1); // Simulate work
4      pthread_mutex_lock(&mutex_B); // Wait for Table B
5      // Critical section
6      pthread_mutex_unlock(&mutex_B);
7      pthread_mutex_unlock(&mutex_A);
8  }
9
10 void transaction2() {
11     pthread_mutex_lock(&mutex_B); // Lock Table B
12     sleep(1); // Simulate work
13     pthread_mutex_lock(&mutex_A); // Wait for Table A
14     // Critical section
15     pthread_mutex_unlock(&mutex_A);
16     pthread_mutex_unlock(&mutex_B);
17 }

```

Mitigation: Enforce a global ordering (e.g., always lock A before B) or use deadlock detection and recovery mechanisms, such as timeouts or resource preemption.

4.5.2 Race Condition Despite Synchronization

In a multithreaded web server, a binary semaphore protects a shared log file, but the critical section is incorrectly defined. The semaphore is released before the file write completes, allowing another thread to access the file concurrently.

WHY? The race condition persists because the critical section is not fully protected. Synchronization tools only work if the entire shared resource operation is atomic. Partial protection or incorrect semaphore placement leads to vulnerabilities.

```

1      sem_t log_mutex;
2  FILE* log_file;
3
4  void log_request(char* request) {
5      sem_wait(&log_mutex);
6      fprintf(log_file, "Request: %s\n", request);
7      sem_signal(&log_mutex);
8      fflush(log_file); // Flushing outside critical section
9  }

```

If another thread starts writing before *fflush* completes, log entries may interleave, corrupting the file.

Mitigation: Ensure the critical section encompasses all operations on the shared resource, including flushing or closing files. Use monitors to encapsulate resource access, reducing the chance of such errors.

4.5.3 Starvation Despite Synchronization

In a reader-writer system using semaphores, readers continuously access a shared database, keeping the writer semaphore blocked. New readers keep arriving, preventing the writer from ever gaining access.

WHY? The semaphore implementation prioritizes readers (e.g., by allowing concurrent reads), but lacks a fairness mechanism to ensure writers get a chance. This is common in systems where one type of process (e.g., readers) has higher throughput or priority.

```
1   sem_t mutex, db;
2   int reader_count = 0;
3
4   void reader() {
5       sem_wait(&mutex);
6       reader_count++;
7       if (reader_count == 1) sem_wait(&db);
8       sem_signal(&mutex);
9       // Read database
10      sem_wait(&mutex);
11      reader_count--;
12      if (reader_count == 0) sem_signal(&db);
13      sem_signal(&mutex);
14  }
15
16  void writer() {
17      sem_wait(&db); // Blocks if readers are active
18      // Write database
19      sem_signal(&db);
20  }
```

If readers keep arriving, *db* remains locked, starving the writer.

Mitigation: Introduce a fairness policy, such as a semaphore to limit the number of consecutive readers or a condition variable in a monitor to prioritize writers after a certain number of reads.

5 Case Study Comparison

5.1 Synchronization in Linux

Linux, a UNIX-like operating system, leverages POSIX standards for synchronization, offering a robust set of tools for managing concurrency in its kernel and user-space applications. Linux emphasizes flexibility and performance, supporting both kernel-level and user-level synchronization.

1. Semaphores:

- Linux provides POSIX semaphores (*sem_t*) for user-space synchronization and kernel semaphores for kernel-space. POSIX semaphores include named and unnamed variants, supporting both mutual exclusion and condition synchronization.
- **Example:** In a Linux web server (e.g., Apache), POSIX semaphores manage a thread pool handling HTTP requests. A counting semaphore limits concurrent connections to prevent resource exhaustion, while a binary semaphore protects a shared log file.

2. Mutexes:

- POSIX mutexes (*pthread_mutex_t*) are used in user-space for thread synchronization. In the kernel, spinlocks and mutexes (e.g., *struct mutex*) protect critical sections.

- **Example:** The Linux kernel uses mutexes to protect access to the process table during fork operations, ensuring that process creation is thread-safe.

3. Monitors:

- Linux does not natively provide monitors but emulates them using POSIX mutexes and condition variables (*pthread_cond_t*). High-level languages like Java running on Linux use monitors for thread-safe objects.
- **Example:** A Linux-based database application might use a monitor-like construct (via *pthread_mutex_t* and *pthread_cond_t*) to manage concurrent transactions, ensuring atomic updates to a shared table.

4. Other Tools:

- **Read-Write Locks:** Linux supports read-write locks (*pthread_rwlock_t*) for scenarios like the reader-writer problem, allowing multiple readers but exclusive writers.
- **Futexes:** Fast user-space mutexes (futexes) optimize synchronization by minimizing kernel involvement, used in high-performance applications like Nginx.

5.1.1 Concurrency handling

- **Kernel-Level:** Linux uses a preemptive kernel, allowing interrupts to preempt kernel execution, which requires fine-grained synchronization. Spinlocks are used for short critical sections, while mutexes handle longer ones.
- **User-Level:** POSIX threads (pthreads) provide a standard API for thread creation and synchronization, widely used in applications like GIMP or MySQL.
- **Example Scenario:** In the Linux Virtual Memory subsystem, semaphores manage page faults. When multiple processes access a shared memory region, a semaphore ensures exclusive updates to the page table, preventing race conditions.

5.1.2 Strengths and Challenges

- **Strengths:** Linux's POSIX-compliant tools are portable and flexible, supporting a wide range of applications. Futexes and spinlocks optimize performance for specific use cases.
- **Challenges:** Complex synchronization in the kernel (e.g., managing interrupts) can lead to deadlocks if not carefully designed. User-space developers must manually ensure fairness to avoid starvation.

5.2 Synchronization in Windows

Windows, a hybrid kernel-based operating system, provides a rich set of synchronization primitives through its Win32 API, designed for both user and kernel space. Windows emphasizes ease of use and integration with its threading model, catering to a broad range of applications from desktop software to servers.

1. Semaphores:

- Windows offers semaphore objects (*CreateSemaphore*) for both mutual exclusion and resource counting. They are kernel objects, accessible across processes.
- **Example:** In a Windows print spooler, a counting semaphore limits the number of active print jobs, ensuring that only a fixed number of printers are used concurrently.

2. Mutexes:

- Windows mutexes (*CreateMutex*) are kernel objects that enforce mutual exclusion, supporting inter-process synchronization. They are similar to POSIX mutexes but include features like ownership tracking.

- **Example:** In a Windows file-sharing application, a mutex protects a shared configuration file, ensuring that only one process modifies it at a time.

3. Monitors:

- Windows does not provide native monitors but supports monitor-like behavior through critical sections (*CRITICAL_SECTION*) and condition variables (*CONDITION_VARIABLE*) in user-space. These are lightweight compared to kernel objects.
- **Example:** A Windows-based banking application might use a critical section and condition variable to implement a monitor for account transactions, ensuring thread-safe deposits and withdrawals.

4. Other Tools:

- **Events:** Windows event objects (*CreateEvent*) signal conditions, similar to condition variables, used for thread coordination.
- **Reader-Writer Locks:** Slim reader-writer locks (*SRWLOCK*) optimize scenarios where multiple readers can access a resource concurrently.
- **Interlocked Operations:** Atomic operations (e.g., *InterlockedIncrement*) provide low-overhead synchronization for simple variables.

5.2.1 Concurrency handling

- **Kernel-Level:** Windows uses a hybrid kernel with preemption, requiring synchronization for kernel objects like handles. Dispatcher objects (e.g., semaphores, mutexes) manage kernel-level concurrency.
- **User-Level:** Windows threads, created via *CreateThread*, integrate seamlessly with Win32 synchronization primitives, used in applications like Microsoft SQL Server.
- **Example Scenario:** In the Windows File System, a mutex protects metadata updates during file creation. When multiple processes write to the same directory, the mutex ensures atomic updates, preventing corruption.

5.2.2 Strengths and Challenges

- **Strengths:** Windows' synchronization primitives are user-friendly, with extensive documentation and integration with tools like Visual Studio. Kernel objects support cross-process synchronization effectively.
- **Challenges:** Kernel-based primitives (e.g., mutexes) can be slower than user-space alternatives due to system call overhead. Improper use of events or locks can lead to starvation or deadlocks.

5.3 Key Differences

- **Portability:** Linux's POSIX-compliant tools are portable across UNIX-like systems, while Windows' Win32 API is proprietary, tailored to its ecosystem.
- **Granularity:** Linux offers fine-grained control (e.g., futexes, spinlocks), ideal for performance-critical applications. Windows prioritizes ease of use with kernel objects and lightweight user-space options.
- **Cross-Process Support:** Windows' kernel objects (e.g., semaphores, mutexes) are designed for inter-process synchronization, while Linux relies on named semaphores or shared memory for similar functionality.
- **Performance Trade-offs:** Linux's futexes minimize kernel involvement, enhancing performance. Windows' critical sections are efficient for user-space, but kernel objects are slower due to system calls.

5.4 Example: Producer-Consumer Problem

- **Linux:** Uses POSIX semaphores to manage a shared buffer in a multimedia application. A binary semaphore (mutex) protects buffer access, while counting semaphores (empty, full) coordinate producers (e.g., video encoders) and consumers (e.g., renderers).
- **Windows:** Implements the same problem using Win32 semaphores. A semaphore object ensures exclusive buffer access, and two others track empty and filled slots. The implementation is similar but uses *WaitForSingleObject* and *ReleaseSemaphore* instead of *sem_wait* and *sem_post*.

6 Visualization

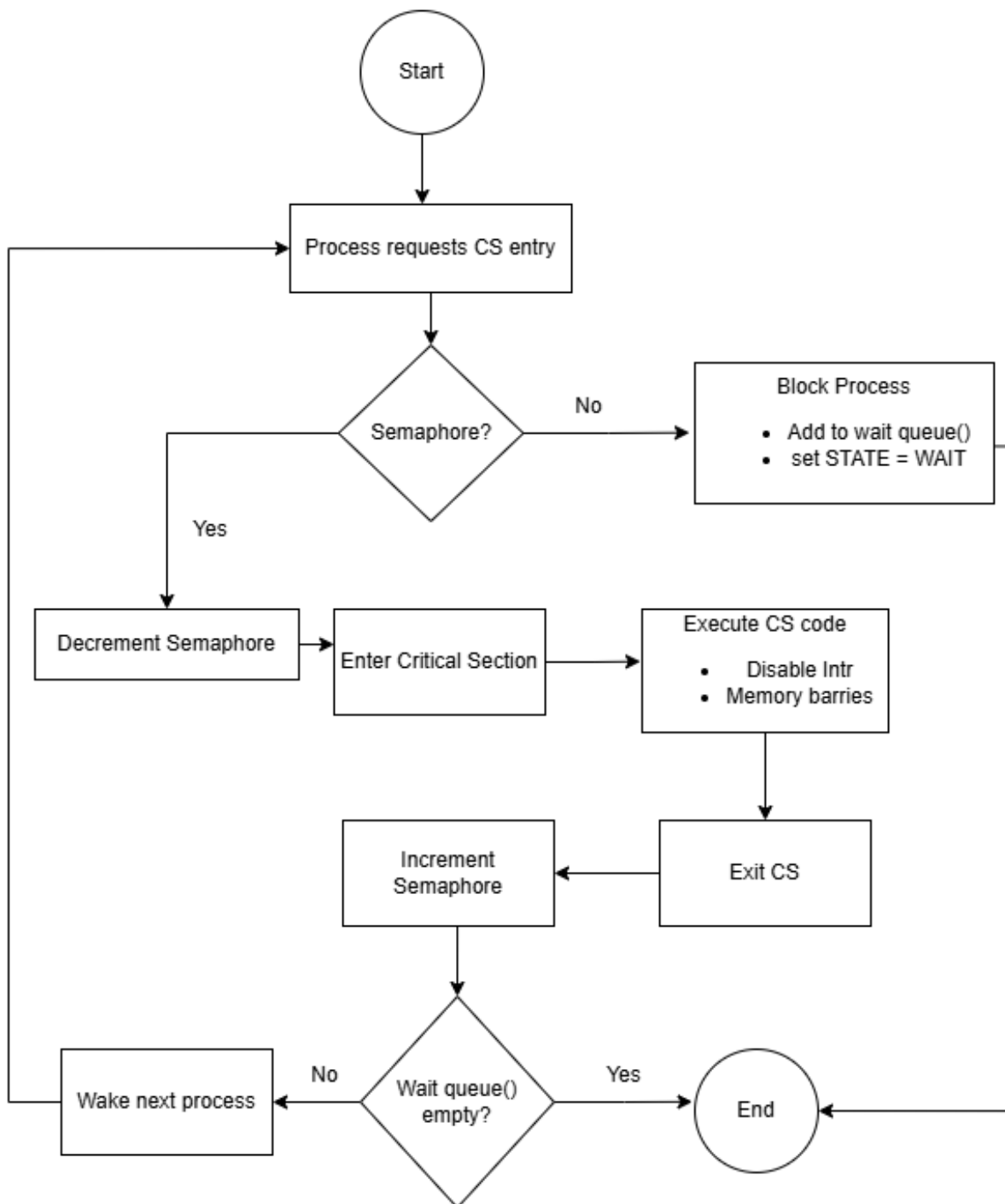


Figure 1: Semaphore in multi-process environment

7 Conclusion

The assignment on synchronization in operating systems, underscores the critical role of synchronization tools—semaphores, mutexes, and monitors—in managing concurrency and ensuring system reliability. Key learnings include understanding how semaphores (binary and counting) coordinate resource access, as demonstrated in the producer-consumer and reader-writer problems; how mutexes enforce mutual exclusion to prevent race conditions; and how monitors simplify synchronization through encapsulated logic. The comparison of Linux and Windows highlights their distinct approaches—Linux’s POSIX flexibility versus Windows’ user-friendly Win32 primitives—while emphasizing universal challenges like deadlock, race conditions, and starvation. Synchronization tools are vital for process coordination, preventing data corruption, and maintaining system stability in multi-process environments. By orchestrating access to shared resources, these tools ensure predictable, efficient, and robust operation, forming the backbone of modern operating systems’ ability to handle complex, concurrent workloads.